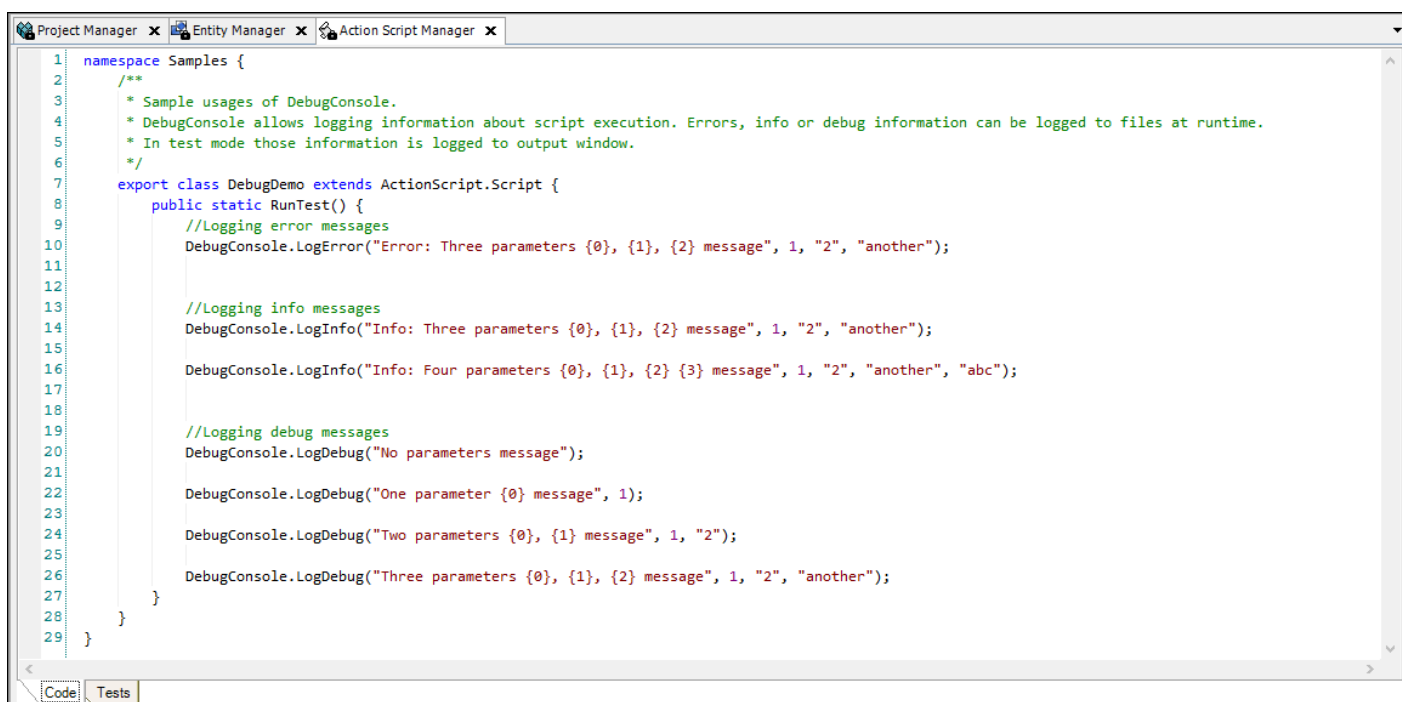


Action Script Function Definition

After adding a new Action Script (via the **Entity Explorer Right-Click Menu**), the **Code** tab is automatically filled with sample code which presents the structure of an Action Script and makes it easier to start writing.



```

1 namespace Samples {
2     /**
3      * Sample usages of DebugConsole.
4      * DebugConsole allows logging information about script execution. Errors, info or debug information can be logged to files at runtime.
5      * In test mode those information is logged to output window.
6      */
7     export class DebugDemo extends ActionScript.Script {
8         public static RunTest() {
9             //Logging error messages
10            DebugConsole.LogError("Error: Three parameters {0}, {1}, {2} message", 1, "2", "another");
11
12
13            //Logging info messages
14            DebugConsole.LogInfo("Info: Three parameters {0}, {1}, {2} message", 1, "2", "another");
15
16            DebugConsole.LogInfo("Info: Four parameters {0}, {1}, {2} {3} message", 1, "2", "another", "abc");
17
18
19            //Logging debug messages
20            DebugConsole.LogDebug("No parameters message");
21
22            DebugConsole.LogDebug("One parameter {0} message", 1);
23
24            DebugConsole.LogDebug("Two parameters {0}, {1} message", 1, "2");
25
26            DebugConsole.LogDebug("Three parameters {0}, {1}, {2} message", 1, "2", "another");
27        }
28    }
29 }
  
```

Action Script Function

An Action Script Function is every public static function in a class that is marked as "export" and extends the `ActionScript.Script` class. Additionally, a function's Inputs and Outputs can only belong to such data types as String, Boolean, Date, Integer, Decimal, and Complex. They can be either scalar or of the list type. The Action Script Function name consists of the namespace, class name, and function name.

Defining Action Script Functions in which the method names differ only by case sensitivity is not permitted.

The Inputs and Outputs have to be declared at the beginning so that they can be used in the body of the function.

Example

The example below contains two Action Script Functions: `Samples.Demo.NoParametersFunction` and `Samples.Demo.ParametersFunction`.

```
namespace Samples {
    export class Demo extends ActionScript.Script {
        public static NoParametersFunction() : void {
        }
        public static ParametersFunction(input1: integer) : void {
        }
    }
}
```

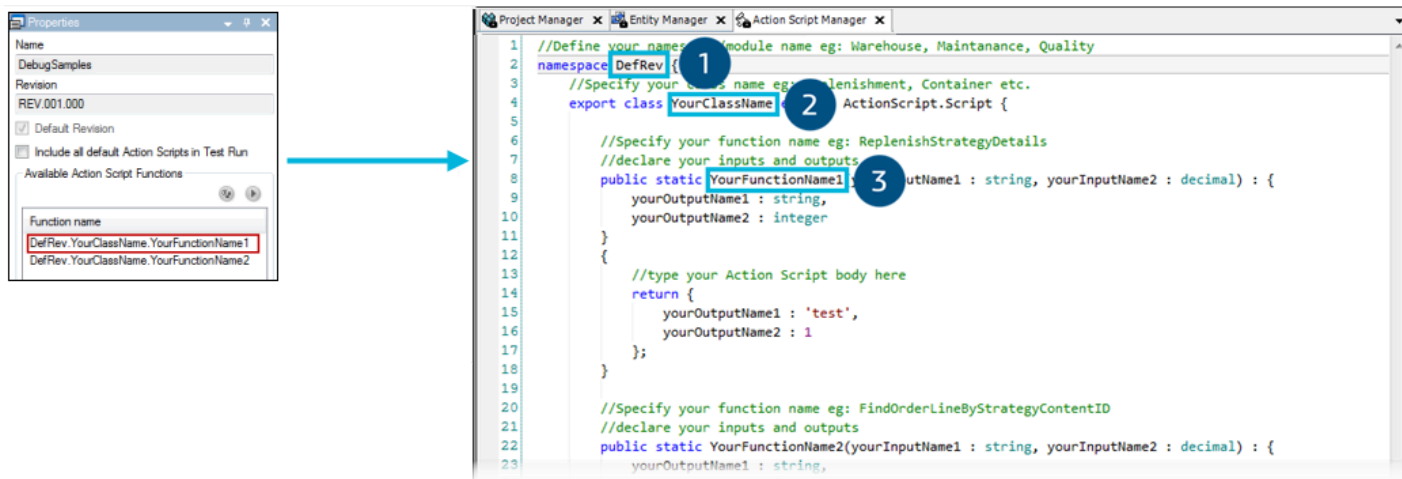
Perform the following steps to write the Action Script code:

1. Define your namespace/module name (e.g., Warehouse, Maintenance, Quality). Each Action Script must start with a namespace.
2. Define a class that you can name as necessary. The class has to be public (export keyword) so that DELMIA Apriso can use it, and it has to extend the ActionScript.Script class.
3. Define the functions inside each class.
4. Declare the function Inputs and Outputs.
5. Enter the Action Script body.

Action Script Function Name

The Action Script Function name consists of the namespace, class name, and function name. For example:

- ▶ Namespace: "DefRev"
- ▶ Class name: "YourClassName"
- ▶ Function name: "YourFunctionName1"



If you have renamed a function in an Action Script whose status is above Design in Progress, you will be asked, upon attempting to save changes, if you want to update references to this function in Screen Flow entities (see item 2. and the following in [Renaming Action Script References](#)).

Action Script Function Inputs and Outputs

Inputs

Only standard data types are supported as Action Script Function Inputs. If any complex type (excluding Date) is used as an Input, the function will not be detected and an Action Script validation will display an error.

Outputs

Outputs can be returned in the following ways:

- Returns a value:

Code	Description
<pre>public static SimpleInputOutput(input: string) : number</pre>	An Output named "ScriptFunctionOutput" of number type will be visible in Process Builder.
<pre>@OutputName("OutputFromScript") public static SimpleInputOutput(input: string) : number</pre>	An Output named "OutputFromScript" of number type will be visible in Process Builder.

- Anonymous type – the function below will have two Outputs: "Out1" and "Out2" of the string and decimal types.

```
public static SomeFunction() : {
    Out1: string,
    Out2: decimal
}
```

- Returns a specific type/class – the function below has three Outputs: "OutM1", "OutM2", and "OutM3".

```
class OutputClass {
    OutM1: string;
    OutM2: boolean;
    OutM3: ComplexObject;
}
public static SomeFunction() : OutputClass
```

Data Types

Action Script Type	Process Builder Type
string	Char
string[]	List of Char
integer	Integer
integer[]	List of Integer
decimal	Decimal
decimal[]	List of Decimal
boolean	Boolean
boolean[]	List of Boolean
date	DateTime
date[]	List of DateTime
ComplexObject	Complex
ComplexObject[]	List of Complex

Number Usage Limitation

Because Action Scripts are created using the TypeScript language which is “compiled” to JavaScript, usage of the Decimal and Integer data type in these scripts is limited.

JavaScript has `number` data type which can store both Decimal and Integer values. Number data type has different precision which may lead to incorrect conversion between JavaScript type and Process Builder type. In order to correctly process numbers with higher precision, use the C# User Formula instead.

The same limitation applies to the decimal data type in the JScript User Formula.

DateTime Usage

The DELMIA Apriso database stores dates (in most cases) in the UTC format. An Action Script with an Input of the DateTime type gets the data (from the database or System Parameters) as a Date object in the UTC format.

This means that if a script uses local DateTime variables, these variables should be created in the correct way (as objects in the UTC format). Creating DateTime variables in other ways can result in issues during compilation and runtime.

To create local variable of the DateTime data type, use one of these methods:

- For a UTC date:

```
var createdOn = new Date(Date.UTC(2016, 1, 29, 0, 50, 0));
```

- For a local time date:

```
var createdOnLocal = new Date(2016, 01, 29, 0, 50, 0);
```

The values for months in the JavaScript and TypeScript languages range from 0 (January) to 11 (December).

Asynchronous Action Scripts

Action Scripts can return a Promise object that can be resolved after a script is run. However, any further processing (for example, Screen Flow Action processing) will happen only after the promise is settled (fulfilled or rejected). The functionality makes it possible to use asynchronous API such as the API described in [Offline Mode API Documentation](#).

For more information on Promise objects, refer to Mozilla Developer Network (MDN) documentation website.

Inside an Action Script Function

Inside the Action Script Function, you can use the following:

- Inputs and Outputs
- Static System Parameters
- The TypeScript language for writing logic
- DELMIA Apriso Functions (like Query) (for details, see [Action Script API: Querying Database](#))

- ▶ Errors – an Action Script Function can display an error (for details, see [Action Scripts API: Raising Error From Scripts](#))
- ▶ Logging functions (for details, see [Action Script API: Logging Debug, Info and Error Messages](#))